CSE242 Assignment 3

Nidhi Shah

Q1

(a)

```
import numpy as np
import matplotlib.pyplot as plt

# Training dataset
X = np.array([
    [1, 1],
    [1, 2],
    [2, 1],
    [0, 0],
    [1, 0],
    [0, 1]
])

y = np.array([1, 1, 1, -1, -1, -1])

print("Training data:")
print(X)
print("Labels:", y)
```

```
Training data:
[[1 1]
 [1 2]
 [2 1]
 [0 0]
 [1 0]
 [0 1]]
Labels: [ 1  1  1 -1 -1 -1]
```

```
def plot_svm_data():
    plt.figure(figsize=(7, 6))
```

```
    # Plot positive and negative points
    plt.scatter(X[y == 1, 0], X[y == 1, 1],
                color='blue', marker='o', s=100, label='Class +1')

    plt.scatter(X[y == -1, 0], X[y == -1, 1],
                color='orange', marker='s', s=100, label='Class -1')

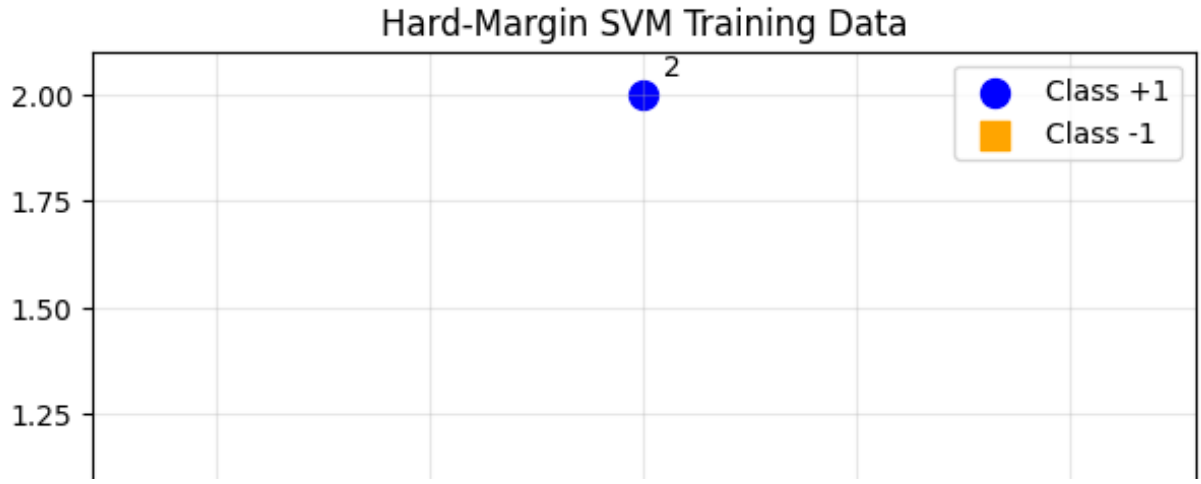
    # Label each point with its ID
    for i, point in enumerate(X):
        plt.annotate(str(i + 1), point,
                     xytext=(7, 7), textcoords='offset points')

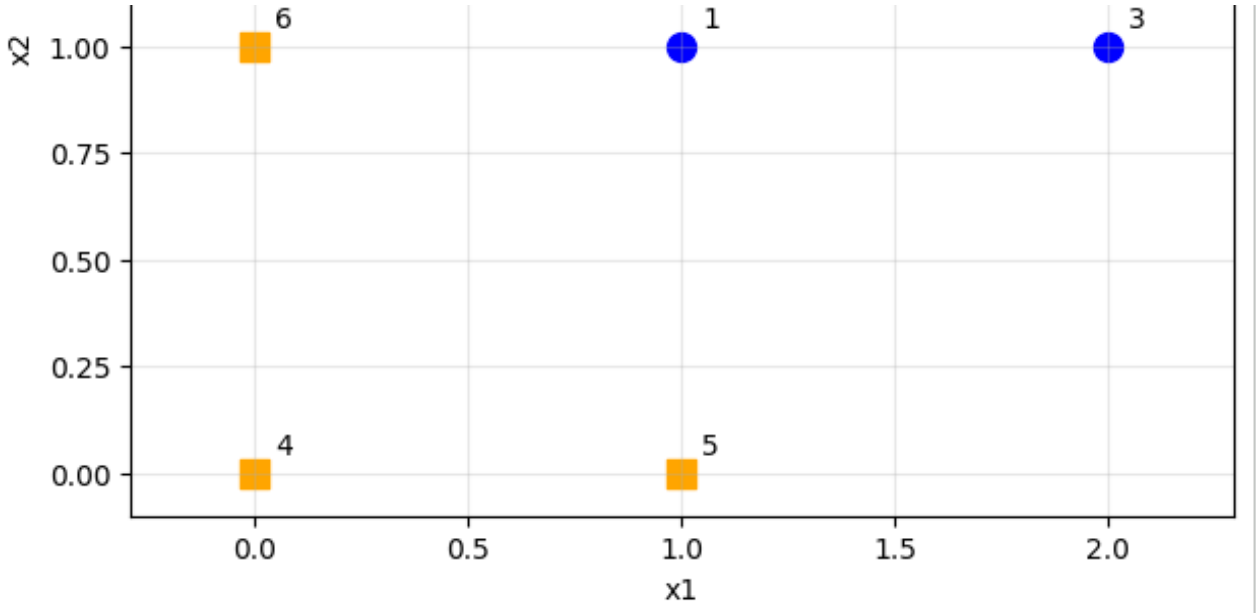
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.title("Hard-Margin SVM Training Data")
    plt.xlim(-0.5, 2.5)
    plt.ylim(-0.5, 2.5)
    plt.grid(alpha=0.3)
    plt.legend()
    plt.axis('equal')
    plt.show()


def identify_support_vectors():
    # Support vector point IDs (1-based)
    return (1, 5, 6)


plot_svm_data()

support_vectors = identify_support_vectors()
print("Support vector point IDs:", support_vectors)
```



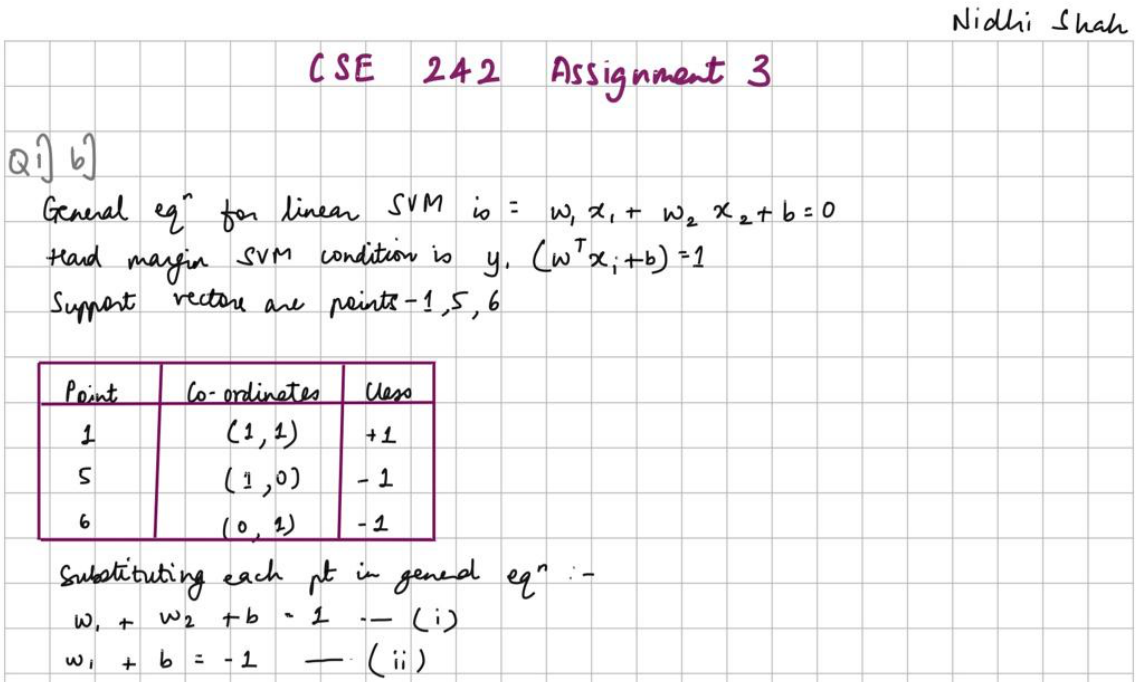



```
Support vector point IDs: (1, 5, 6)
```

(b)

Insert Derivation here

```
from IPython.display import display, Image
display(Image(filename= 'Q1b.jpeg'))
```



Nidhi Shah

CSE 242 Assignment 3

Q1] b]

General eq^n for linear SVM is = $w_1 x_1 + w_2 x_2 + b = 0$

Hard margin SVM condition is $y_i (w^T x_i + b) = 1$

Support vectors are points - 1, 5, 6

Point	Co-ordinates	Class
1	(1, 1)	+1
5	(1, 0)	-1
6	(0, 1)	-1

Substituting each pt in general eq^n :-

$w_1 + w_2 + b = 1$ — (i)

$w_1 + b = -1$ — (ii)

$w_2 + b = -1 \quad \text{--- (iii)}$

(iii) - (ii) $= w_1 = w_2$

Substituting in (i) & (ii) $= 2w_1 + b = 1 \text{ ---- (iv)}$

$w_1 + b = -1 \text{ --- (v)}$

(iv) - (v) $= w_1 = 2$

$\therefore \quad w_2 = 2 \rightarrow b = -3$

$\therefore \quad 2x_1 + 2x_2 - 3 = 0$ final decision boundary

Now, margin $= \frac{1}{\|w\|}$

$\|w\| = \sqrt{w_1^2 + w_2^2} = \sqrt{2^2 + 2^2}$

$= \sqrt{8} = 2\sqrt{2}$

$\therefore$ Margin $= \frac{1}{2\sqrt{2}}$

$\therefore$ Margin $= 0.354$

Full margin width $= \frac{2}{\|w\|} = 2 * (0.354)$

Full margin width $= 0.707$

```
def derive_svm_separator():

    # Weights and bias obtained from support vector equations
    w = np.array([2.0, 2.0])
    b = -3.0

    # Calculate geometric margin
    geometric_margin = 1 / np.linalg.norm(w)

    return w, b, geometric_margin


weights, bias, margin = derive_svm_separator()

print("Weights:", weights)
print("Bias:", bias)
print("Decision boundary: 2x1 + 2x2 - 3 = 0")
print("Geometric margin:", margin)
print("Full margin width:", 2 * margin)
```

```
Weights: [2. 2.]
Bias: -3.0
Decision boundary: 2x1 + 2x2 - 3 = 0
Geometric margin: 0.35355339059327373
Full margin width: 0.7071067811865475
```

```
plt.figure(figsize=(7, 6))

# Plot training points
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            color='blue', marker='o', s=100, label='Class +1')
```

```
plt.scatter(X[y == -1, 0], X[y == -1, 1],
            color='orange', marker='s', s=100, label='Class -1')

# Label points
for i, point in enumerate(X):
    plt.annotate(str(i + 1), point,
                 xytext=(7, 7), textcoords='offset points')

# Generate x1 values
x1 = np.linspace(-0.5, 2.5, 200)

# Decision boundary: 2x1 + 2x2 - 3 = 0
decision_boundary = (3 - 2 * x1) / 2

# Positive supporting boundary: 2x1 + 2x2 - 3 = 1
positive_boundary = (4 - 2 * x1) / 2

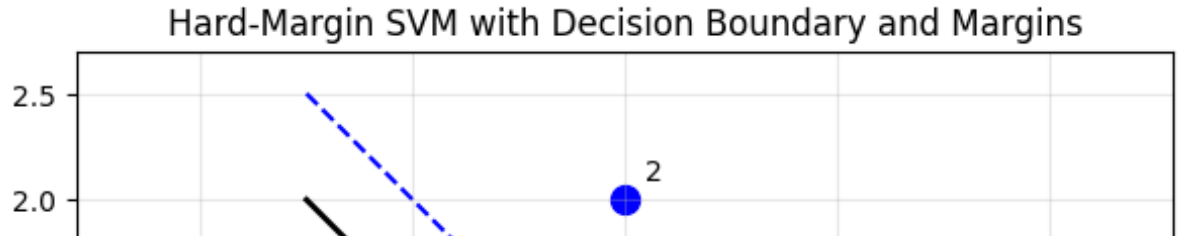
# Negative supporting boundary: 2x1 + 2x2 - 3 = -1
negative_boundary = (2 - 2 * x1) / 2

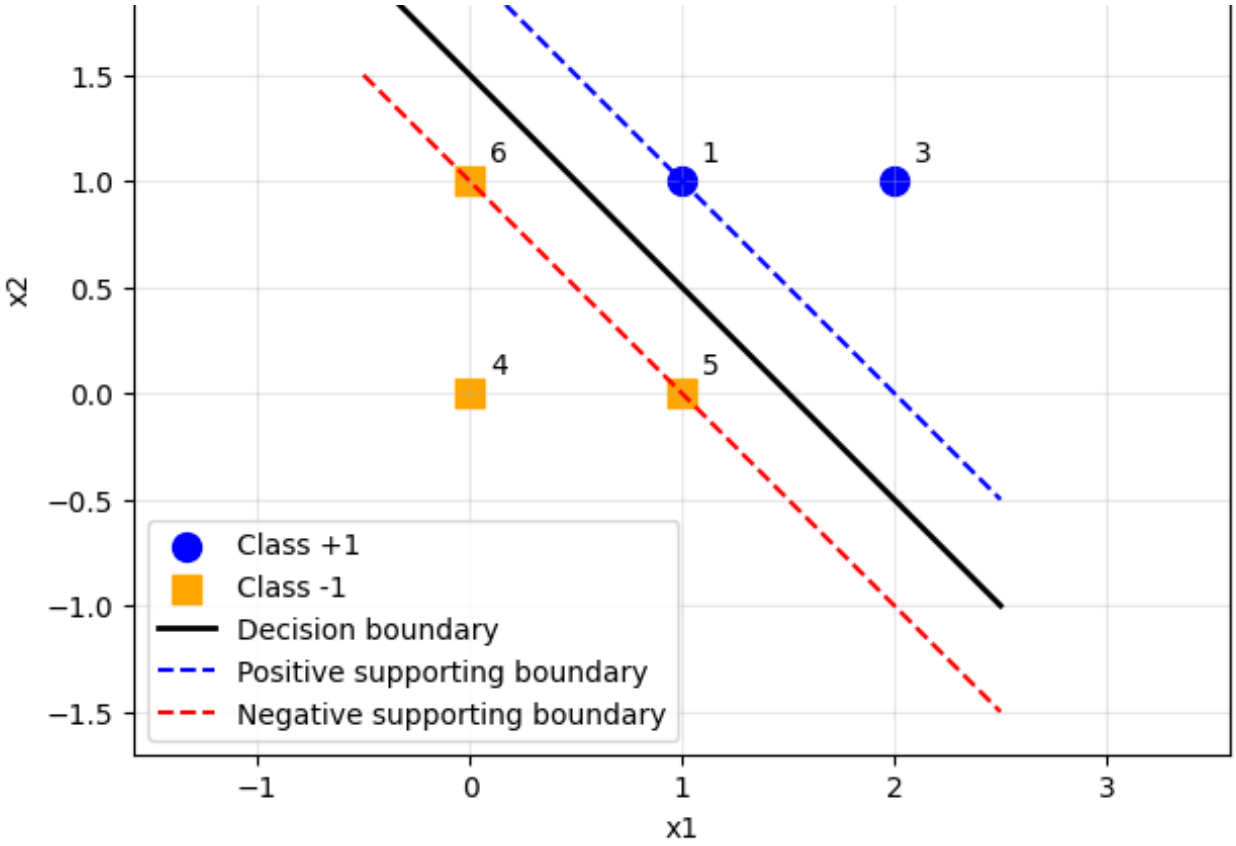
plt.plot(x1, decision_boundary, 'k-', linewidth=2,
         label='Decision boundary')

plt.plot(x1, positive_boundary, 'b--',
         label='Positive supporting boundary')

plt.plot(x1, negative_boundary, 'r--',
         label='Negative supporting boundary')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Hard-Margin SVM with Decision Boundary and Margins")
plt.xlim(-0.5, 2.5)
plt.ylim(-0.5, 2.5)
plt.grid(alpha=0.3)
plt.legend()
plt.axis('equal')
plt.show()
```




1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
x2
−1
0
1
2
3
x1
6
1
3
4
5
Class +1
Class -1
Decision boundary
Positive supporting boundary
Negative supporting boundary

```
print("========== Q1 FINAL RESULTS ==========")

print("Support Vectors:", identify_support_vectors())

print("Weight w1:", weights[0])
print("Weight w2:", weights[1])
print("Bias b:", bias)

print("Decision Boundary:")
print(f"{weights[0]}x1 + {weights[1]}x2 + ({bias}) = 0")

print("Geometric Margin:", round(margin, 4))
print("Full Margin Width:", round(2 * margin, 4))
```

```
========== Q1 FINAL RESULTS ==========
Support Vectors: (1, 5, 6)
Weight w1: 2.0
Weight w2: 2.0
Bias b: -3.0
Decision Boundary:
2.0x1 + 2.0x2 + (-3.0) = 0
Geometric Margin: 0.3536
Full Margin Width: 0.7071
```

Q2

(a)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate the given toy dataset
X, true_labels = make_blobs(
    n_samples=4000,
    centers=4,
    cluster_std=0.60,
    random_state=0
)

print("Dataset shape:", X.shape)
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
```

```
Dataset shape: (4000, 2)
Number of samples: 4000
Number of features: 2
```

```
def update_assignments(data, centroids):
    """
    Assign each data point to its nearest centroid.
    In case of equal distances, choose the first centroid.
    """

    # Calculate squared Euclidean distances
    distances = np.sum(
        (data[:, np.newaxis, :] - centroids[np.newaxis, :, :]) ** 2,
        axis=2
    )

    # Assign each point to the nearest centroid
    assignments = np.argmin(distances, axis=1)

    return assignments


def update_centroids(data, centroids, assignments):
    """
    Update centroids by calculating the mean of assigned points.
    If a cluster is empty, retain its previous centroid.
    """

    new_centroids = centroids.copy()

    for k in range(len(centroids)):

        # Find points belonging to the current cluster
        cluster_points = data[assignments == k]

        # Update centroid if cluster is not empty
        if len(cluster_points) > 0:
            new_centroids[k] = np.mean(cluster_points, axis=0)

        # Otherwise retain the previous centroid

    return new_centroids


print("K-means update functions implemented successfully.")
```

```
K-means update functions implemented successfully.
```

K-means clustering works by repeatedly assigning each data point to its nearest centroid and updating the centroids based on the mean of the points assigned to each cluster. I implemented both update steps using NumPy without using a library K-means implementation. For equal distances, the point is assigned to the first centroid. If a cluster becomes empty, its previous centroid is retained.

How we handle special cases:

- Distance ties: np.argmin() selects the first centroid with the minimum distance.
- Empty clusters: The previous centroid is retained instead of calculating the mean of an empty cluster.

```
def kmeans(data, centroids, max_iterations=100):

    for _ in range(max_iterations):

        # Step 1: Assign points to nearest centroids
        assignments = update_assignments(data, centroids)

        # Step 2: Update centroid positions
        centroids = update_centroids(data, centroids, assignments)

    # Perform final assignment using updated centroids
    assignments = update_assignments(data, centroids)

    return centroids, assignments


print("K-means algorithm implemented successfully.")
```

```
K-means algorithm implemented successfully.
```

```
K_VALUES = (2, 3, 4, 6, 10)

def run_experiments():

    # Generate the dataset
    data, true_labels = make_blobs(
        n_samples=4000,
        centers=4,
```

```
            cluster_std=0.60,
            random_state=0
        )

        # Random generator for reproducible initialization
        rng = np.random.default_rng(0)

        results = {}

        for k in K_VALUES:

            # Initialize centroids by randomly selecting data points
            initial_centroids = data[
                rng.integers(0, data.shape[0], size=k)
            ].copy()

            # Run K-means
            final_centroids, assignments = kmeans(
                data, initial_centroids
            )

            results[k] = (final_centroids, assignments)

            print(f"\nResults for k = {k}")
            print("Final centroids:")
            print(final_centroids)

            print("Cluster sizes:")
            print(np.bincount(assignments, minlength=k))

        return data, results


    data, results = run_experiments()
```

```
Results for k = 2
Final centroids:
[[-1.21305404  7.77779068]
 [ 0.48431531  2.68854595]]
Cluster sizes:
[1017 2983]

Results for k = 3
Final centroids:
[[ 2.01786097  0.91940356]
```

```
 [-0.29364657  3.6083596 ]
 [-1.23802329  7.81533456]]
Cluster sizes:
[1008 1992 1000]

Results for k = 4
Final centroids:
[[-1.23802329  7.81533456]
 [ 2.01811513  0.89810959]
 [-1.52380199  2.91767743]
 [ 0.96183057  4.29478497]]
Cluster sizes:
[1000  998 1002 1000]

Results for k = 6
Final centroids:
[[-1.23802329  7.81533456]
 [-1.62303717  3.60890155]
 [ 2.01995361  0.8977222 ]
 [-1.91528189  2.55418093]
 [-0.82928446  2.78221954]
 [ 0.98621239  4.30822173]]
Cluster sizes:
[1000  299  997  407  319  978]

Results for k = 10
Final centroids:
[[-1.67167458  8.19344295]
 [-0.63530957  7.92568855]
 [ 2.11237375  1.49183312]
 [ 2.22322923  0.32933759]
 [ 0.95587054  4.31327241]
 [-1.53118398  2.92273293]
 [ 1.20395473  1.57525361]
 [ 1.41741161  0.57069555]
 [-1.42282007  7.19556804]
 [ 2.82764677  0.97229097]]
Cluster sizes:
[365 350 253 261 988 997 123 213 285 165]
```

```
def plot_clusters(data, results):

    for k, (centroids, assignments) in results.items():

        plt.figure(figsize=(8, 6))

        # Plot data points based on their cluster assignments
```

```
        plt.scatter(
            data[:, 0],
            data[:, 1],
            c=assignments,
            cmap='viridis',
            s=15,
            alpha=0.6
        )

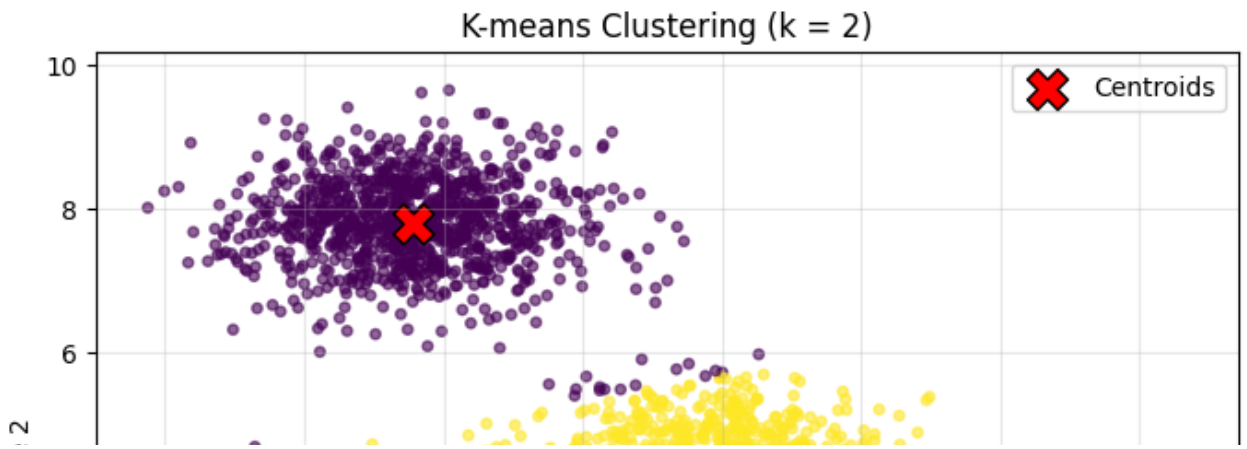
        # Plot final centroids
        plt.scatter(
            centroids[:, 0],
            centroids[:, 1],
            color='red',
            marker='X',
            s=250,
            edgecolors='black',
            label='Centroids'
        )

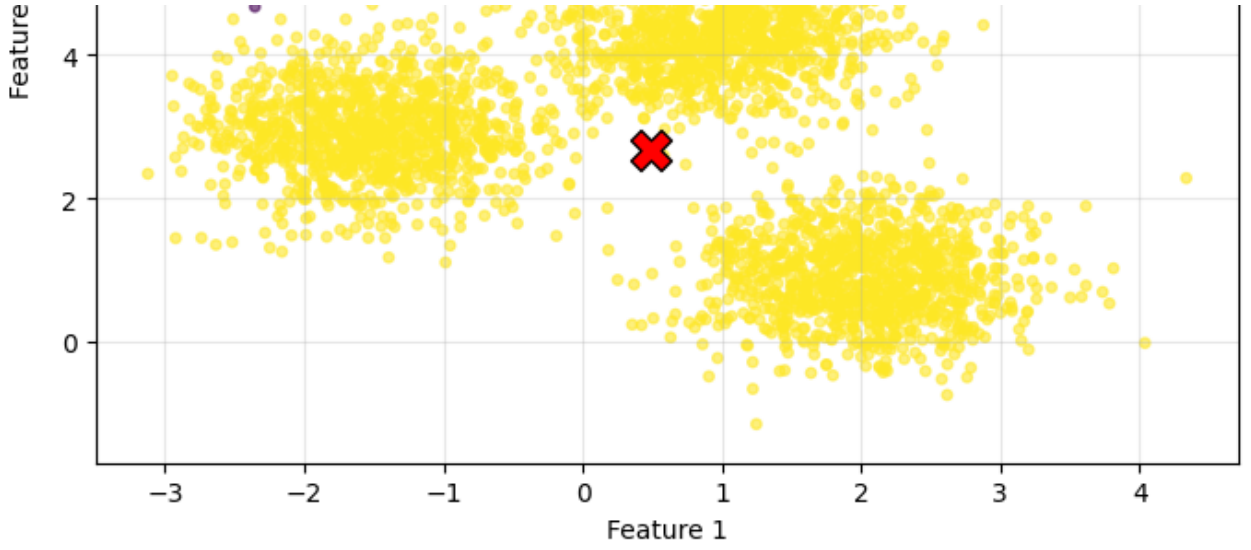
        plt.title(f"K-means Clustering (k = {k})")
        plt.xlabel("Feature 1")
        plt.ylabel("Feature 2")
        plt.legend()
        plt.grid(alpha=0.3)

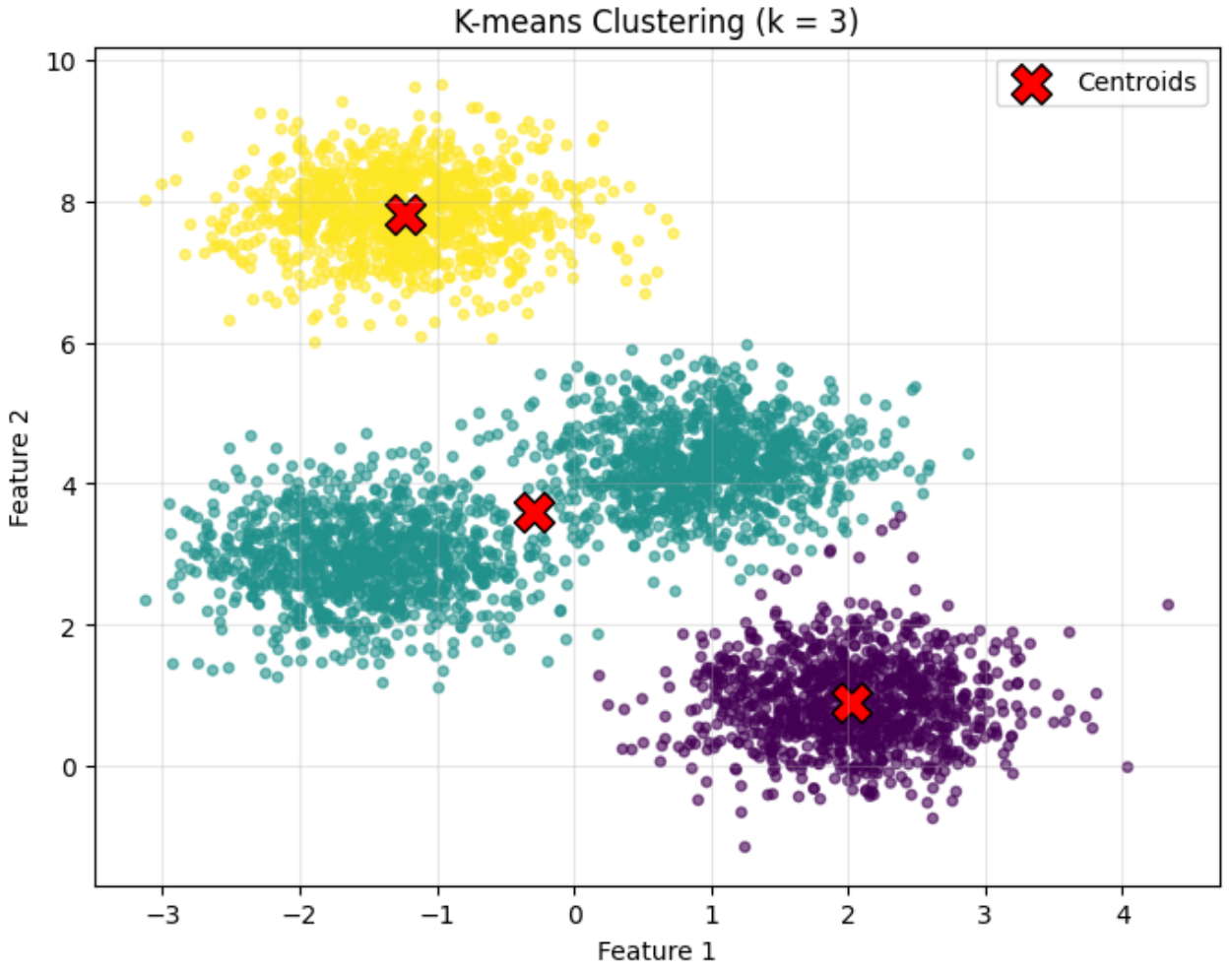
        plt.savefig(f"kmeans_k{k}.png", dpi=150,
                    bbox_inches='tight')

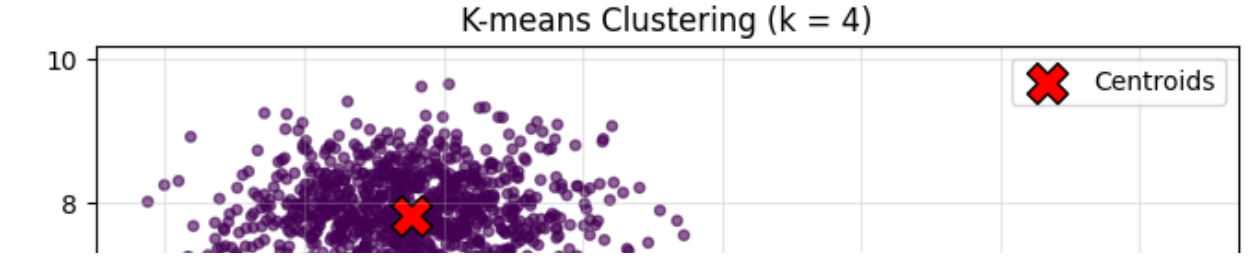
        plt.show()

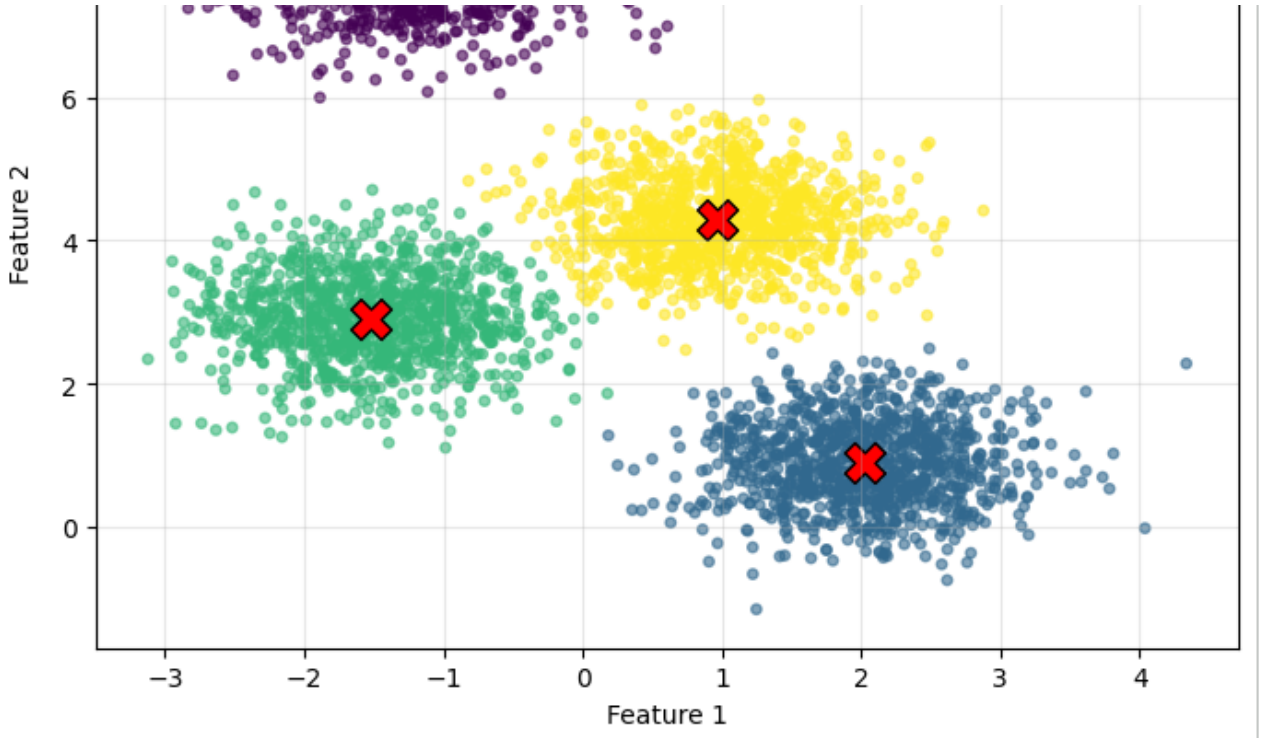

plot_clusters(data, results)
```

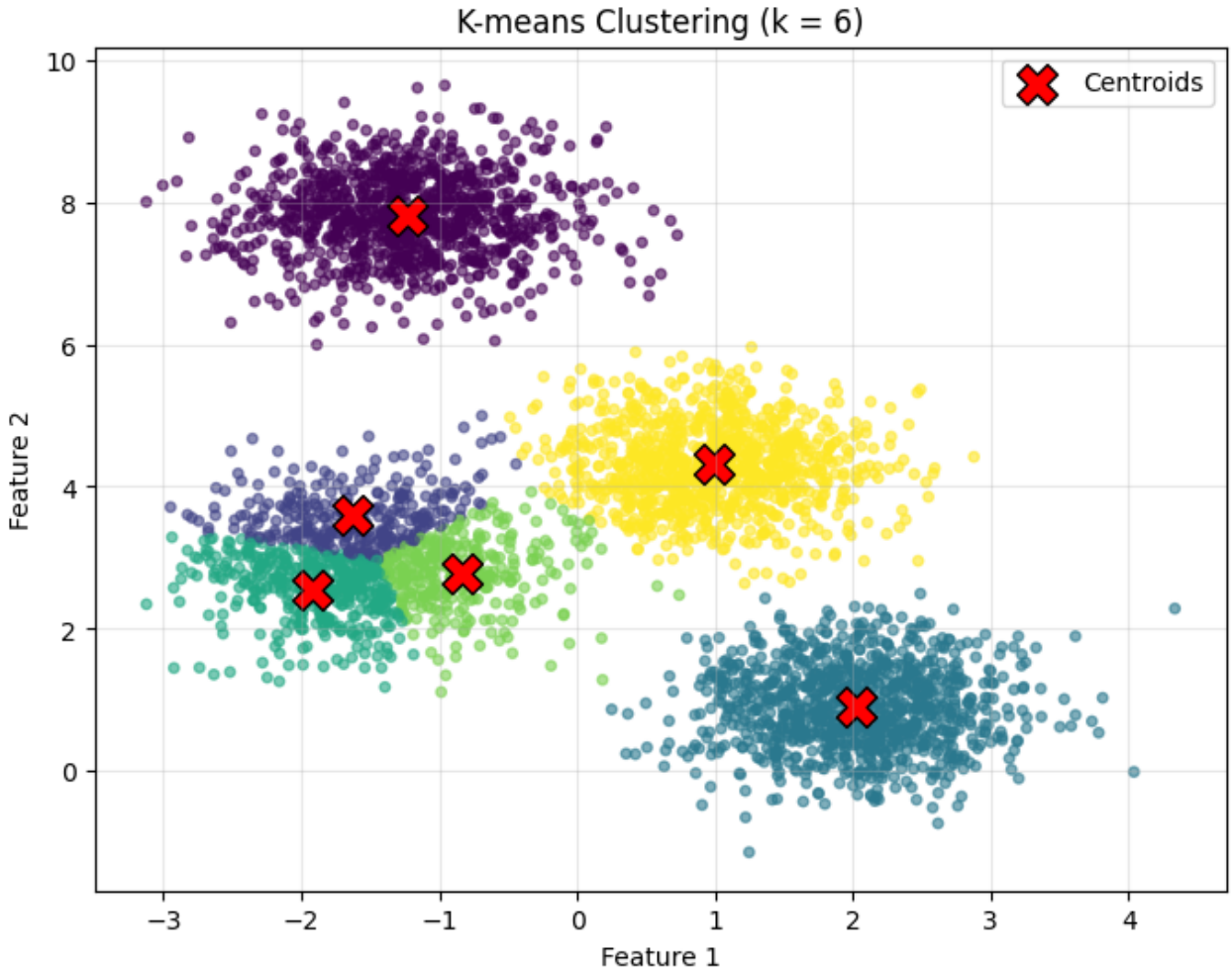



Feature
4
2
0
−3
−2
−1
0
1
2
3
4
Feature 1

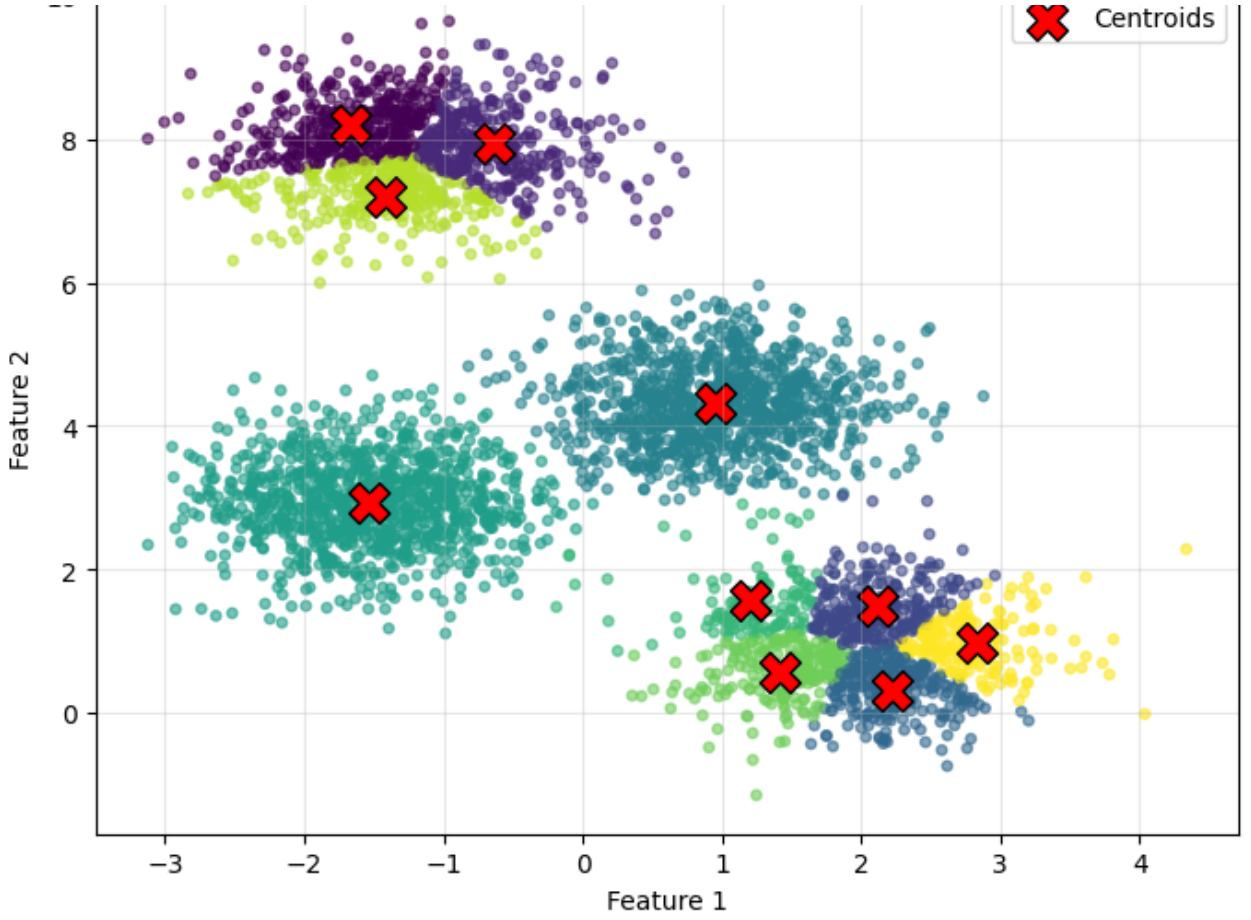

K-means Clustering (k = 3)
Centroids
10
8
6
4
2
0
Feature 2
−3
−2
−1
0
1
2
3
4
Feature 1


K-means Clustering (k = 4)
Centroids
10
8


Feature 2
Feature 1


K-means Clustering (k = 6)
Centroids
Feature 2
Feature 1


K-means Clustering (k = 10)




```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
```

```
for ax, k in zip(axes.flat, K_VALUES):

    centroids, assignments = results[k]

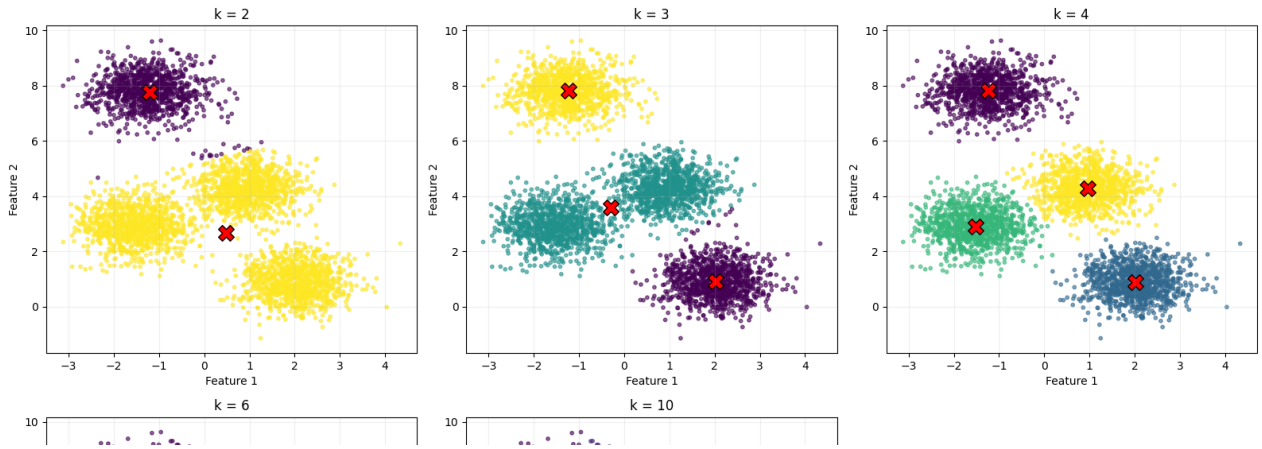
    ax.scatter(
        data[:, 0],
        data[:, 1],
        c=assignments,
        cmap='viridis',
        s=10,
        alpha=0.6
    )

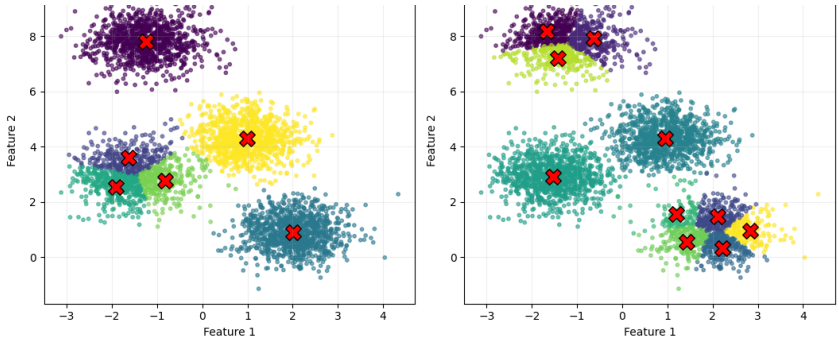
    ax.scatter(
        centroids[:, 0],
        centroids[:, 1],
        color='red',
        marker='X',
        s=200,
        edgecolors='black'
    )

    ax.set_title(f"k = {k}")
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.grid(alpha=0.2)

# Hide unused subplot
axes.flat[5].axis('off')

plt.tight_layout()
plt.show()
```





As the value of k increases, the dataset is divided into more clusters, resulting in smaller and more detailed groups. When k = 2, the four original generating groups are combined into two larger clusters. When k = 3, some original groups are still merged. At k = 4, the clusters generally correspond to the four original generating groups. When k = 6 and k = 10, some original groups are further divided into smaller clusters. This shows that increasing k creates more clusters, but does not necessarily mean better clustering because the actual dataset was generated using four groups.

Q3

(a)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.neighbors import NearestNeighbors

# SVM parameters
SVM_C = 1.0
SVM_GAMMA = 0.01

# Label flipping probabilities
POSITIVE_FLIP_PROBABILITY = 0.35
NEGATIVE_FLIP_PROBABILITY = 0.20
```

```
def generate_clean_split(rng, samples_per_class=1000):

    # Positive class: mean (1,1), covariance I
    positive = rng.multivariate_normal(
        mean=(1, 1),
        cov=np.eye(2),
        size=samples_per_class
    )

    # Negative class: mean (-1,-1), covariance 3I
    negative = rng.multivariate_normal(
        mean=(-1, -1),
        cov=3 * np.eye(2),
        size=samples_per_class
    )

    # Combine both classes
    features = np.vstack((positive, negative))

    labels = np.concatenate((
        np.ones(samples_per_class, dtype=int),
        -np.ones(samples_per_class, dtype=int)
    ))

    return features, labels


# Generate independent training and testing sets
rng = np.random.default_rng(0)

X_train, y_train_clean = generate_clean_split(rng)
X_test, y_test = generate_clean_split(rng)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (2000, 2)
Testing data shape: (2000, 2)
```

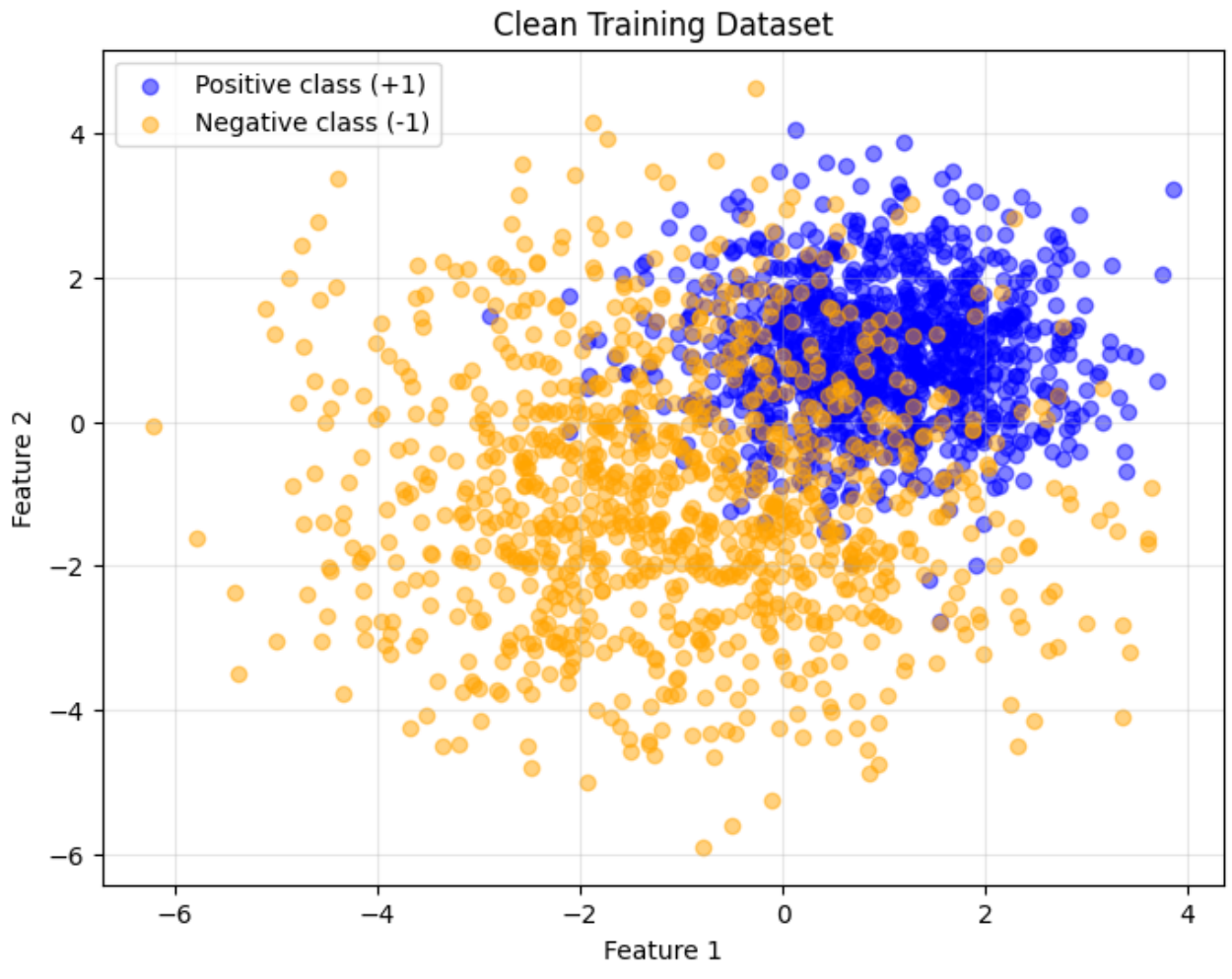
```
plt.figure(figsize=(8, 6))

plt.scatter(
    X_train[y_train_clean == 1, 0],
    X_train[y_train_clean == 1, 1],
```

```
    color='blue',
    label='Positive class (+1)',
    alpha=0.5
)

plt.scatter(
    X_train[y_train_clean == -1, 0],
    X_train[y_train_clean == -1, 1],
    color='orange',
    label='Negative class (-1)',
    alpha=0.5
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Clean Training Dataset")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
def corrupt_training_labels(clean_labels, rng):

    # Generate a new copy of original labels
    noisy_labels = clean_labels.copy()

    # Generate independent random values for each sample
    random_values = rng.random(len(clean_labels))

    # Flip positive labels with probability 0.35
    positive_mask = clean_labels == 1

    positive_flips = (
        positive_mask &
        (random_values < POSITIVE_FLIP_PROBABILITY)
    )

    # Flip negative labels with probability 0.20
    negative_mask = clean_labels == -1

    negative_flips = (
        negative_mask &
        (random_values < NEGATIVE_FLIP_PROBABILITY)
    )

    # Change the selected labels
    noisy_labels[positive_flips | negative_flips] *= -1

    return noisy_labels


# Test label corruption
rng_test = np.random.default_rng(42)

y_train_noisy = corrupt_training_labels(
    y_train_clean, rng_test
)

print("Original labels:", np.unique(y_train_clean, return_counts=True
print("Noisy labels:", np.unique(y_train_noisy, return_counts=True))
```

```
print(
    "Number of flipped labels:",
    np.sum(y_train_clean != y_train_noisy)
)
```

```
Original labels: (array([-1,  1]), array([1000, 1000]))
Noisy labels: (array([-1,  1]), array([1171,  829]))
Number of flipped labels: 551
```

```
def train_and_evaluate(
    train_features,
    train_labels,
    test_features,
    test_labels
):

    # Create RBF SVM
    model = SVC(
        kernel='rbf',
        C=SVM_C,
        gamma=SVM_GAMMA
    )

    # Train using given training labels
    model.fit(train_features, train_labels)

    # Predict on clean test data
    predictions = model.predict(test_features)

    # Calculate accuracy
    accuracy = accuracy_score(test_labels, predictions)

    return accuracy


# Train baseline SVM using noisy training labels
baseline_accuracy = train_and_evaluate(
    X_train,
    y_train_noisy,
    X_test,
    y_test
)

print("Baseline SVM Accuracy:", baseline_accuracy)
```

```
Baseline SVM Accuracy: 0.7985
```

The training dataset contains 1,000 positive and 1,000 negative examples generated from two Gaussian distributions. I independently introduced label noise by flipping positive labels with probability 0.35 and negative labels with probability 0.20. I trained an RBF SVM using C = 1 and gamma = 0.01. I repeated the experiment 20 times while keeping the training and testing features fixed and evaluated the model on clean test labels.

(b)

```
def repair_labels_knn(train_features, noisy_labels, neighbors=5):

    # Calculate pairwise Euclidean distances
    differences = (
        train_features[:, np.newaxis, :] -
        train_features[np.newaxis, :, :]
    )

    distances = np.sqrt(np.sum(differences ** 2, axis=2))

    # Exclude each point itself
    np.fill_diagonal(distances, np.inf)

    # Find the five nearest neighbors
    # Stable sorting resolves equal-distance ties by row index
    nearest_indices = np.argsort(
        distances, axis=1, kind='stable'
    )[:, :neighbors]

    # Get neighbors' labels from the original noisy-label array
    neighbor_labels = noisy_labels[nearest_indices]

    # Majority voting
    repaired_labels = np.where(
        np.sum(neighbor_labels, axis=1) > 0,
        1,
        -1
    )

    return repaired_labels
```

```
# Test K-NN label repair
y_train_repaired = repair_labels_knn(
    X_train,
    y_train_noisy,
    neighbors=5
)

print("Noisy labels:", np.unique(y_train_noisy, return_counts=True))
print("Repaired labels:", np.unique(y_train_repaired, return_counts=T

print(
    "Labels changed during repair:",
    np.sum(y_train_noisy != y_train_repaired)
)
```

```
Noisy labels: (array([-1,  1]), array([1171,  829]))
Repaired labels: (array([-1,  1]), array([1253,  747]))
Labels changed during repair: 764
```

```
def run_comparison(repeats=20, seed=0, neighbors=5,
                   output_path="noise_comparison.csv"):

    # Generate independent clean training and testing sets
    rng = np.random.default_rng(seed)

    X_train, y_train_clean = generate_clean_split(rng)
    X_test, y_test = generate_clean_split(rng)

    results = []

    for trial in range(1, repeats + 1):

        # Step 1: Corrupt the original training labels
        noisy_labels = corrupt_training_labels(
            y_train_clean, rng
        )

        # Step 2: Baseline SVM using noisy labels
        baseline_accuracy = train_and_evaluate(
            X_train,
            noisy_labels,
            X_test,
            y_test
        )
```

```
        # Step 3: Repair labels using K-NN
        repaired_labels = repair_labels_knn(
            X_train,
            noisy_labels,
            neighbors
        )

        # Step 4: Train a fresh SVM using repaired labels
        knn_accuracy = train_and_evaluate(
            X_train,
            repaired_labels,
            X_test,
            y_test
        )

        # Store results for this trial
        results.append({
            "trial": trial,
            "baseline_accuracy": baseline_accuracy,
            "knn_accuracy": knn_accuracy
        })

        print(
            f"Trial {trial:02d}: "
            f"Baseline = {baseline_accuracy:.4f}, "
            f"K-NN = {knn_accuracy:.4f}"
        )

    # Convert results into a DataFrame
    results_df = pd.DataFrame(results)

    # Save results as CSV
    results_df.to_csv(output_path, index=False)

    # Calculate mean accuracies
    mean_baseline = results_df["baseline_accuracy"].mean()
    mean_knn = results_df["knn_accuracy"].mean()

    difference = mean_knn - mean_baseline

    print("\n========== FINAL RESULTS ==========")

    print(f"Mean Baseline Accuracy: {mean_baseline:.4f}")
    print(f"Mean K-NN Accuracy: {mean_knn:.4f}")
```

```
    print(f"Accuracy Difference: {difference:+.4f}")

    print(f"\nResults saved to: {output_path}")

    return results_df, mean_baseline, mean_knn, difference


results_df, mean_baseline, mean_knn, difference = run_comparison(
    repeats=20,
    seed=0,
    neighbors=5,
    output_path="noise_comparison.csv"
)
```

```
Trial 01: Baseline = 0.8345, K-NN = 0.8615
Trial 02: Baseline = 0.8330, K-NN = 0.8720
Trial 03: Baseline = 0.8440, K-NN = 0.8675
Trial 04: Baseline = 0.8445, K-NN = 0.8700
Trial 05: Baseline = 0.8395, K-NN = 0.8725
Trial 06: Baseline = 0.8410, K-NN = 0.8715
Trial 07: Baseline = 0.8405, K-NN = 0.8690
Trial 08: Baseline = 0.7930, K-NN = 0.8480
Trial 09: Baseline = 0.8240, K-NN = 0.8615
Trial 10: Baseline = 0.8425, K-NN = 0.8715
Trial 11: Baseline = 0.8220, K-NN = 0.8600
Trial 12: Baseline = 0.8190, K-NN = 0.8645
Trial 13: Baseline = 0.8470, K-NN = 0.8645
Trial 14: Baseline = 0.8415, K-NN = 0.8720
Trial 15: Baseline = 0.8160, K-NN = 0.8715
Trial 16: Baseline = 0.8285, K-NN = 0.8485
Trial 17: Baseline = 0.8235, K-NN = 0.8695
Trial 18: Baseline = 0.8430, K-NN = 0.8700
Trial 19: Baseline = 0.7840, K-NN = 0.8335
Trial 20: Baseline = 0.7980, K-NN = 0.8690

========== FINAL RESULTS ==========
Mean Baseline Accuracy: 0.8280
Mean K-NN Accuracy: 0.8644
Accuracy Difference: +0.0364

Results saved to: noise_comparison.csv
```

```
display(results_df)
```

	trial	baseline_accuracy	knn_accuracy
0	1	0.8345	0.8615
1	2	0.8330	0.8720
2	3	0.8440	0.8675
3	4	0.8445	0.8700
4	5	0.8395	0.8725
5	6	0.8410	0.8715
6	7	0.8405	0.8690
7	8	0.7930	0.8480
8	9	0.8240	0.8615
9	10	0.8425	0.8715
10	11	0.8220	0.8600
11	12	0.8190	0.8645
12	13	0.8470	0.8645
13	14	0.8415	0.8720
14	15	0.8160	0.8715
15	16	0.8285	0.8485
16	17	0.8235	0.8695
17	18	0.8430	0.8700
18	19	0.7840	0.8335
19	20	0.7980	0.8690

Next steps: Generate code with `results_df` New interactive sheet

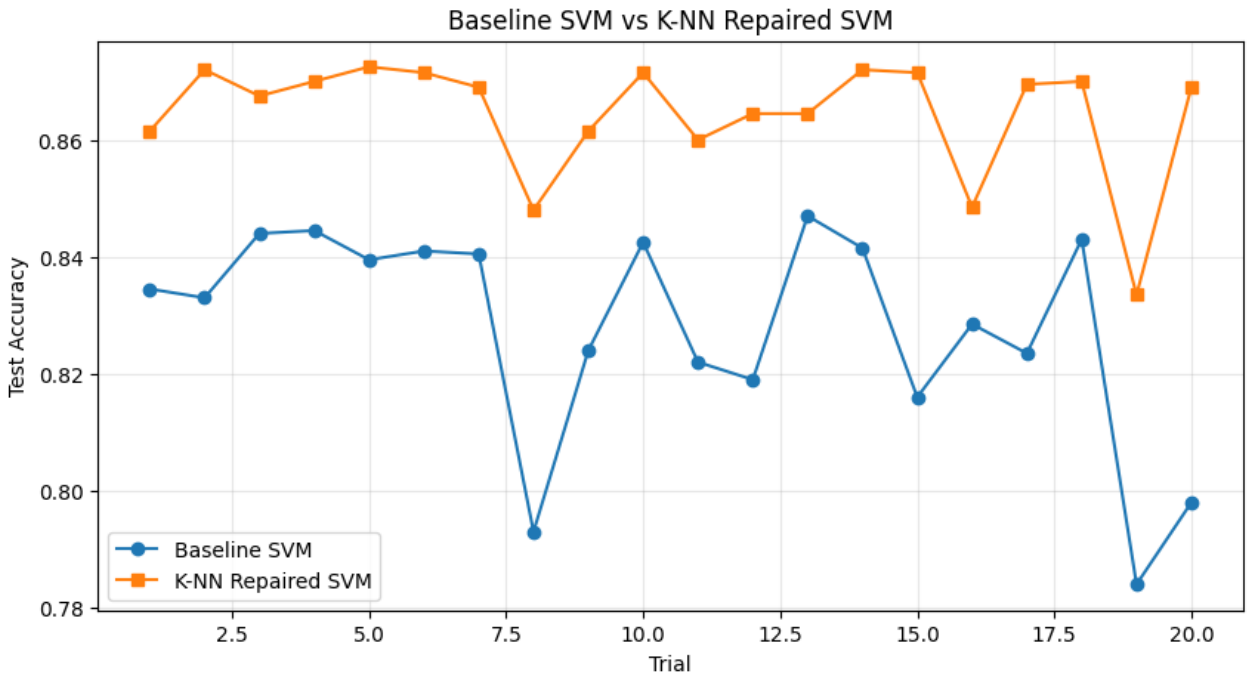
```
plt.figure(figsize=(10, 5))

plt.plot(
```

```
        results_df["trial"],
        results_df["baseline_accuracy"],
        marker='o',
        label='Baseline SVM'
    )

    plt.plot(
        results_df["trial"],
        results_df["knn_accuracy"],
        marker='s',
        label='K-NN Repaired SVM'
    )

    plt.xlabel("Trial")
    plt.ylabel("Test Accuracy")
    plt.title("Baseline SVM vs K-NN Repaired SVM")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()
```



```
from google.colab import files

files.download("noise_comparison.csv")
```

I implemented K-NN label repair using the five nearest neighbors of each training point, calculated using Euclidean distance. Each training label was replaced by the majority label of its five nearest neighbors. I excluded the point itself from its neighborhood and used the original noisy labels for all votes to avoid sequential updates. I then trained a fresh RBF SVM using the repaired labels and compared its accuracy with the baseline SVM.

```
Start coding or generate with AI.
```